Akhali Shen — Full AWS Production Deployment Guide

> **Stack:** Next.js 16 (frontend) · Django 5 / Gunicorn (backend) · PostgreSQL (RDS) · Nginx · Docker Compose · Let's Encrypt SSL
> **Last updated:** June 2026

Table of Contents

1. Architecture Overview

```
Internet
   │
   ▼
Route 53 (DNS)
   │
   ▼
EC2 Instance (Ubuntu 24.04)
   └── Docker Compose
         ├── nginx  (ports 80 + 443)   ← entry point, SSL termination
         ├── frontend :3000             ← Next.js / Node
         └── backend  :8000             ← Django / Gunicorn
               │
               └── RDS PostgreSQL :5432  (private VPC, no public access)

Media files:  Docker volume  backend_media  (or S3 – see Section 12)
Static files: served by Whitenoise inside Django container
```

2. AWS Account Preparation

2.1 IAM User for deployments

1. **IAM** → **Users** → **Create user** → name: `axalishen-deploy`
2. Attach managed policies:
   - `AmazonEC2FullAccess`
   - `AmazonRDSFullAccess`
   - `AmazonS3FullAccess` *(only needed if you use S3 for media)*
3. **Security credentials** → **Create access key** → type: CLI → download CSV

2.2 Key Pair (for SSH)

1. **EC2** → **Key Pairs** → **Create key pair**
2. Name: `axalishen-prod` · Type: RSA · Format: `.pem`
3. **Save the downloaded** `.pem` **file** — you cannot re-download it.
4. On Mac/Linux: `chmod 400 axalishen-prod.pem`

3. RDS PostgreSQL Database

1. **RDS** → **Create database**
2. Settings:

Field	Value
Engine	PostgreSQL 15
Template	Production (or Free tier for testing)
DB identifier	`axalishen-db`
Master username	`axalishen`
Master password	Generate a strong one — **save it now**
Initial DB name	`axalishen`
Instance class	`db.t3.micro` (dev) / `db.t3.small` (prod)
Storage	20 GB gp2, enable autoscaling
Public access	**No**
VPC	Default
VPC Security Group	Create new → name: `axalishen-rds-sg`

3. After creation, note the **Endpoint hostname**, e.g.:
   `axalishen-db.c3xxxxxxxxxx.eu-west-1.rds.amazonaws.com`
4. Go to `axalishen-rds-sg` → **Inbound rules** → **Edit** → add:
   - Type: PostgreSQL · Port: 5432 · Source: *the EC2 security group (add after Step 4)*

---

4. EC2 Instance

4.1 Launch

1. **EC2** → **Launch Instance**

Field	Value
Name	`axalishen-prod`
AMI	Ubuntu 24.04 LTS (64-bit)
Instance type	`t3.small` (minimum) / `t3.medium` (recommended)
Key pair	`axalishen-prod`
Storage	20 GB gp3

2. **Security Group** — create `axalishen-ec2-sg`:

Type	Port	Source
SSH	22	My IP only

Type	Port	Source
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0

4.2 Elastic IP (static public IP)

1. **EC2** → **Elastic IPs** → **Allocate Elastic IP**
2. **Associate** → select your `axalishen-prod` instance
3. Note the IP address — this is your permanent server IP.

4.3 Allow EC2 → RDS

Go back to `axalishen-rds-sg` → **Inbound rules** → **Edit**:

- Type: PostgreSQL · Port: 5432 · Source: `axalishen-ec2-sg`

---

5. Domain & SSL

5.1 Route 53

1. **Route 53** → **Hosted zones** → **Create hosted zone**
2. Domain: `yourdomain.ge`
3. Copy the 4 NS records from Route 53 into your domain registrar's DNS settings.
4. Create A records:

Record	Type	Value
`@` (root)	A	Your Elastic IP
`www`	CNAME	`yourdomain.ge`

DNS propagation takes 5–30 minutes.

---

6. Server Setup

SSH into the server:

```
ssh -i axalishen-prod.pem ubuntu@YOUR_ELASTIC_IP
```

6.1 Install Docker

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
newgrp docker
docker --version          # verify
docker compose version    # verify
```

6.2 Install Certbot

```
sudo apt install -y certbot nginx
```

6.3 Get SSL Certificate (before starting Docker)

```
# Certbot temporarily uses system nginx to verify domain ownership
sudo systemctl start nginx
sudo certbot certonly --nginx -d yourdomain.ge -d www.yourdomain.ge
sudo systemctl stop nginx
sudo systemctl disable nginx    # Docker takes over port 80/443
```

Certs are saved at: `/etc/letsencrypt/live/yourdomain.ge/`

---

7. Deploy the Application

7.1 Clone repository

```
sudo mkdir -p /app && sudo chown ubuntu:ubuntu /app
git clone https://github.com/mozaikkonew/axalishen.git /app/axalishen
cd /app/axalishen
```

7.2 Create `backend/.env.prod`

```
nano backend/.env.prod
```

Paste and fill in the values:

```
# Generate SECRET_KEY with:
# python3 -c "import secrets; print(secrets.token_urlsafe(50))"
SECRET_KEY=YOUR_GENERATED_SECRET_KEY_HERE

DEBUG=False

# Your actual domain(s)
ALLOWED_HOSTS=yourdomain.ge,www.yourdomain.ge

# Your actual domain(s) for CORS
CORS_ALLOWED_ORIGINS=https://yourdomain.ge,https://www.yourdomain.ge

# RDS connection string from Step 3
DATABASE_URL=postgres://axalishen:YOUR_DB_PASSWORD@axalishen-db.xxxxx.eu-west-1.rds.amazonaws.com:543
2/axalishen

MEDIA_ROOT=/app/media
```

7.3 Create root `.env` (for Next.js build arg)

```
nano .env
```

```
NEXT_PUBLIC_API_URL=https://yourdomain.ge
```

7.4 Update `nginx/nginx.conf` for HTTPS

Replace the entire file with:

```
worker_processes auto;
events { worker_connections 1024; }

http {
    include       mime.types;
    default_type  application/octet-stream;
    sendfile      on;
    keepalive_timeout 65;
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml;

    upstream frontend { server frontend:3000; }
    upstream backend  { server backend:8000;  }

    # Redirect HTTP → HTTPS
    server {
        listen 80;
        server_name yourdomain.ge www.yourdomain.ge;
        return 301 https://$host$request_uri;
    }

    server {
        listen 443 ssl;
        server_name yourdomain.ge www.yourdomain.ge;

        ssl_certificate     /etc/nginx/ssl/fullchain.pem;
        ssl_certificate_key /etc/nginx/ssl/privkey.pem;
        ssl_protocols       TLSv1.2 TLSv1.3;
        ssl_ciphers         HIGH:!aNULL:!MD5;

        client_max_body_size 20M;

        location /api/ {
            proxy_pass http://backend;
            proxy_set_header Host              $host;
            proxy_set_header X-Real-IP         $remote_addr;
            proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto https;
        }
        location /admin/ {
            proxy_pass http://backend;
            proxy_set_header Host              $host;
            proxy_set_header X-Forwarded-Proto https;
        }
        location /static/  { proxy_pass http://backend; }
        location /media/   { proxy_pass http://backend; }
        location /ckeditor/ { proxy_pass http://backend; }

        location / {
            proxy_pass http://frontend;
            proxy_set_header Host              $host;
            proxy_set_header X-Forwarded-Proto https;
            proxy_http_version 1.1;
            proxy_set_header Upgrade    $http_upgrade;
```

```
            proxy_set_header Connection "upgrade";
        }
    }
}
```

7.5 Mount SSL certs into Docker

```
mkdir -p nginx/ssl
sudo cp /etc/letsencrypt/live/yourdomain.ge/fullchain.pem nginx/ssl/
sudo cp /etc/letsencrypt/live/yourdomain.ge/privkey.pem  nginx/ssl/
sudo chmod 644 nginx/ssl/*.pem
```

7.6 Run Django migrations

```
docker compose -f docker-compose.prod.yml run --rm backend \
  python manage.py migrate
```

7.7 Start all services

```
docker compose -f docker-compose.prod.yml up -d --build
```

Check status:

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f
```

---

8. Migrate Local Data & Images to Production

> This section covers how to export ALL content you have entered in the local Django admin (site settings, hero section, features, stats, blog posts, services, products, team members, FAQs, partners, etc.) and their uploaded images, and import everything on the production server.

What gets migrated

App	Models
`core`	SiteSettings (logo, favicon, phone, address, social links…)
`content`	HeroSection, Feature, Stat, Testimonial, FAQ, Partner, Achievement, Publication
`about`	AboutPage, CompanyValue, TeamMember, CompanyTimeline, Certification
`services`	Service, ServiceFeature

App	Models
`blog`	BlogCategory, BlogPost
`products`	ProductCategory, Product
Images	`media/` folder: hero/, blog/, services/, products/, team/, certifications/, partners/, testimonials/, site/

Step 8.1 — Export database (on your LOCAL machine)

Open a terminal in your local project folder:

```
cd d:\Users\User\Desktop\axalishen

# Export ALL app data (excluding auth/sessions/contenttypes)
py -3 backend/manage.py dumpdata \
  core content about services blog products contact \
  --indent 2 \
  --output backend/data_export.json
```

> **Windows PowerShell alternative** (if `py -3` not found):
>
> ```
> python backend/manage.py dumpdata core content about services blog products contact --indent 2 --
> output backend/data_export.json
> ```

This creates `backend/data_export.json` — a full JSON dump of all your CMS data.

Step 8.2 — Export media/images (on your LOCAL machine)

Zip the entire `backend/media/` folder:

Windows PowerShell:

```
Compress-Archive -Path backend\media\* -DestinationPath backend\media_export.zip
```

Mac/Linux:

```
cd backend && zip -r media_export.zip media/ && cd ..
```

Step 8.3 — Upload both files to the server

From your local machine:

```
# Upload the data dump
scp -i axalishen-prod.pem backend/data_export.json ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/

# Upload the media archive
scp -i axalishen-prod.pem backend/media_export.zip ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/
```

> **Windows PowerShell** (if `scp` not available, use WinSCP or PuTTY SCP):
>
> ```
> scp -i axalishen-prod.pem backend\data_export.json ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/
> scp -i axalishen-prod.pem backend\media_export.zip ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/
> ```

Step 8.4 — SSH into server and import data

```
ssh -i axalishen-prod.pem ubuntu@YOUR_ELASTIC_IP
cd /app/axalishen
```

Import the database dump:

```
docker compose -f docker-compose.prod.yml run --rm \
  -v $(pwd)/backend/data_export.json:/tmp/data_export.json \
  backend \
  python manage.py loaddata /tmp/data_export.json
```

You should see output like:

```
Installed 87 object(s) from 1 fixture(s)
```

Extract and copy media files:

```
# Extract the zip inside the container's media volume
docker compose -f docker-compose.prod.yml run --rm \
  -v $(pwd)/backend/media_export.zip:/tmp/media_export.zip \
  backend \
  bash -c "cd /app && unzip -o /tmp/media_export.zip -d ."
```

This extracts all images into the `backend_media` Docker volume so Django can serve them via `/media/`.

Step 8.5 — Create a superuser on production

```
docker compose -f docker-compose.prod.yml run --rm backend \
  python manage.py createsuperuser
```

Enter your desired admin username, email, and password when prompted.

Step 8.6 — Verify data import

Visit these URLs after deployment:

- `https://yourdomain.ge/api/v1/settings/` → should return your real site settings JSON
- `https://yourdomain.ge/api/v1/hero/` → should return hero content
- `https://yourdomain.ge/api/v1/posts/` → should list blog posts
- `https://yourdomain.ge/admin/` → log in with your superuser credentials

Re-migrating after local changes

Every time you add new content locally and want to sync to production:

```
# 1. On local — export fresh dump
py -3 backend/manage.py dumpdata core content about services blog products contact \
  --indent 2 --output backend/data_export.json

# 2. Zip any new media
Compress-Archive -Path backend\media\* -DestinationPath backend\media_export.zip -Force

# 3. Upload
scp -i axalishen-prod.pem backend/data_export.json ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/
scp -i axalishen-prod.pem backend/media_export.zip ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/

# 4. On server — flush old content and re-import
docker compose -f docker-compose.prod.yml run --rm backend \
  python manage.py flush --no-input

docker compose -f docker-compose.prod.yml run --rm \
  -v $(pwd)/backend/data_export.json:/tmp/data_export.json \
  backend python manage.py loaddata /tmp/data_export.json

docker compose -f docker-compose.prod.yml run --rm \
  -v $(pwd)/backend/media_export.zip:/tmp/media_export.zip \
  backend bash -c "cd /app && unzip -o /tmp/media_export.zip -d ."

# 5. Recreate superuser (flush deleted it)
docker compose -f docker-compose.prod.yml run --rm backend \
  python manage.py createsuperuser
```

---

9. Verify Deployment

URL	Expected result
`https://yourdomain.ge`	Next.js homepage loads
`https://yourdomain.ge/en`	English version
`https://yourdomain.ge/ru`	Russian version

URL	Expected result
`https://yourdomain.ge/admin/`	Django admin login page
`https://yourdomain.ge/api/v1/settings/`	JSON with site settings
`https://yourdomain.ge/api/v1/hero/`	JSON with hero section
`https://yourdomain.ge/api/v1/posts/`	JSON list of blog posts
`https://yourdomain.ge/media/hero/xxx.jpg`	Uploaded image loads

10. SSL Auto-Renewal

Certbot certificates expire every 90 days. Set up automatic renewal:

```
sudo crontab -e
```

Add this line:

```
0 3 * * * certbot renew --quiet && \
  cp /etc/letsencrypt/live/yourdomain.ge/fullchain.pem /app/axalishen/nginx/ssl/ && \
  cp /etc/letsencrypt/live/yourdomain.ge/privkey.pem /app/axalishen/nginx/ssl/ && \
  docker exec $(docker ps -qf "name=nginx") nginx -s reload
```

Test the renewal process works:

```
sudo certbot renew --dry-run
```

11. Useful Management Commands

```
# View live logs for all services
docker compose -f docker-compose.prod.yml logs -f

# View logs for a specific service
docker compose -f docker-compose.prod.yml logs -f backend
docker compose -f docker-compose.prod.yml logs -f frontend
docker compose -f docker-compose.prod.yml logs -f nginx

# Restart a single service
docker compose -f docker-compose.prod.yml restart backend

# Deploy updated code (git pull + rebuild)
git pull
docker compose -f docker-compose.prod.yml up -d --build

# Run Django management commands
docker compose -f docker-compose.prod.yml exec backend python manage.py shell
docker compose -f docker-compose.prod.yml exec backend python manage.py migrate
docker compose -f docker-compose.prod.yml exec backend python manage.py createsuperuser
docker compose -f docker-compose.prod.yml exec backend python manage.py collectstatic --noinput

# Take a database backup
docker compose -f docker-compose.prod.yml exec backend \
  python manage.py dumpdata --indent 2 --output /app/backup_$(date +%Y%m%d).json

# Copy backup to local machine
scp -i axalishen-prod.pem \
  ubuntu@YOUR_ELASTIC_IP:/app/axalishen/backend/backup_*.json ./
```

12. Optional: S3 for Media Files

Using S3 means uploaded images survive instance replacements and can be served via CloudFront CDN.

12.1 Create S3 Bucket

1. **S3** → **Create bucket** → name: `axalishen-media` → region: same as EC2
2. **Block Public Access**: uncheck "Block all public access"
3. **Bucket policy** → add:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::axalishen-media/*"
  }]
}
```

12.2 Add to `requirements.txt`

```
django-storages[s3]>=1.14
boto3>=1.34
```

12.3 Add to `backend/config/settings.py`

```
import os
if not DEBUG:
    DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
    AWS_STORAGE_BUCKET_NAME = 'axalishen-media'
    AWS_S3_REGION_NAME = 'eu-west-1'
    AWS_S3_CUSTOM_DOMAIN = f'{AWS_STORAGE_BUCKET_NAME}.s3.amazonaws.com'
    MEDIA_URL = f'https://{AWS_S3_CUSTOM_DOMAIN}/'
```

12.4 Add to `backend/.env.prod`

```
AWS_ACCESS_KEY_ID=your-iam-access-key
AWS_SECRET_ACCESS_KEY=your-iam-secret-key
```

12.5 Sync existing media to S3

```
aws s3 sync backend/media/ s3://axalishen-media/ --acl public-read
```

---

13. Cost Estimate (monthly, EU region)

Service	Spec	Est. Cost
EC2 t3.small	On-demand, 24/7	~$15/mo
RDS db.t3.micro	PostgreSQL 15, 20GB	~$15/mo
Elastic IP	(free when attached)	$0
Route 53	1 hosted zone	$0.50/mo

Service	Spec	Est. Cost
Data transfer	~10 GB/mo	~$1/mo
S3 + CloudFront	media files (optional)	~$2/mo
SSL (Let's Encrypt)	Free	$0
Total		**~$33/mo**

Low-cost option: Use `t3.micro` + SQLite instead of RDS = ~$10/mo total.
Switch to RDS when you need: backups, multi-AZ, read replicas, or separate DB scaling.

Quick Reference

```
Repo (personal/Vercel):  https://github.com/Rafael1444/axalishenverceltest
Repo (org):              https://github.com/mozaikkonew/axalishen
Vercel preview:          https://axalishen.vercel.app  (frontend only)
Production:              https://yourdomain.ge          (full stack on EC2)

Django admin:            https://yourdomain.ge/admin/
API base:                https://yourdomain.ge/api/v1/

Docker services:
  nginx     → port 80 / 443 (public)
  frontend  → port 3000 (internal)
  backend   → port 8000 (internal)
  RDS       → port 5432 (private VPC only)
```